session 2

Table of contents

01 interrupts

02 Timer’s mode

03 ATmega328p Timers

04 Timer delays

05 Servos & PWM

interrupts: what are they?

the polling vs interrupts:

A microcontroller has one job execute instructions, one by one, non stop.
But it doesn't live in isolation. It lives in the physical world where buttons get pressed, sensors trigger, timers expire, signals arrive. All of this happening outside the CPU, at any moment, unpredictably.

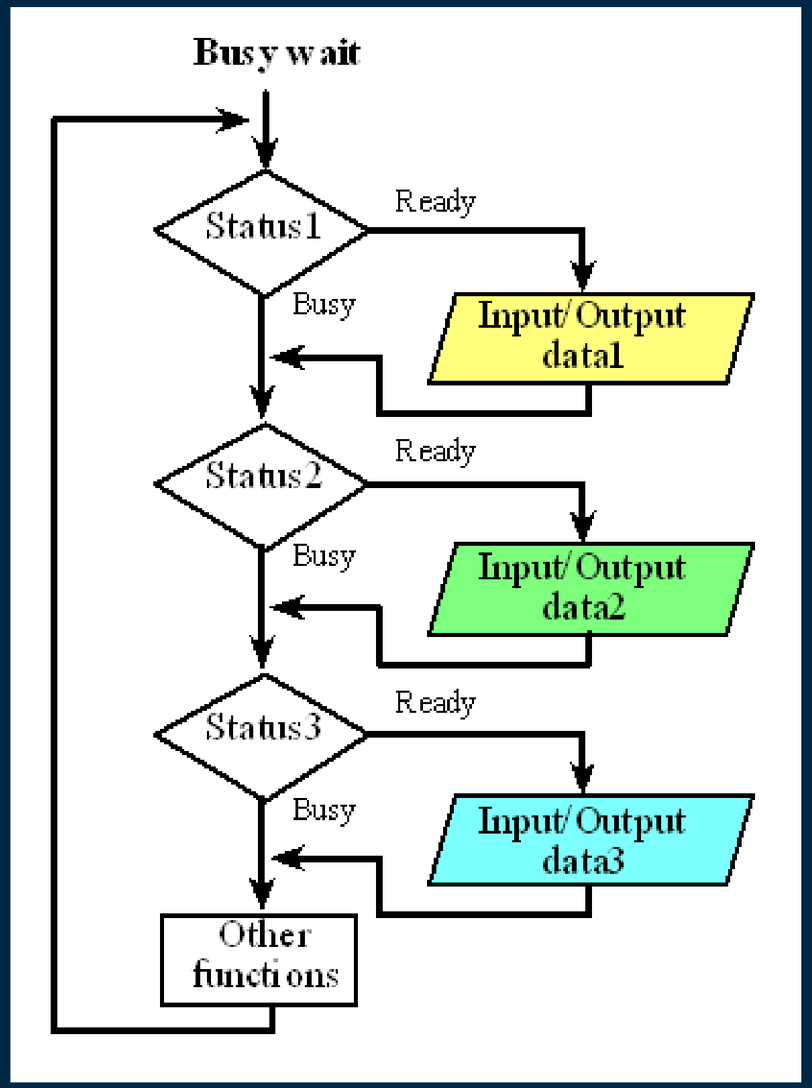
The question is: how does the CPU know when these events happen?

There are two answers: Polling and Interrupts.

interrupts: what are they?

the polling vs interrupts:

In polling, in order for the CPU to react to any external event a sensor triggered, a status changed... it has to constantly loop through every device and check their status one by one.The CPU will waste CPU cycles in the process of checking, and potentially cause a delay in reacting



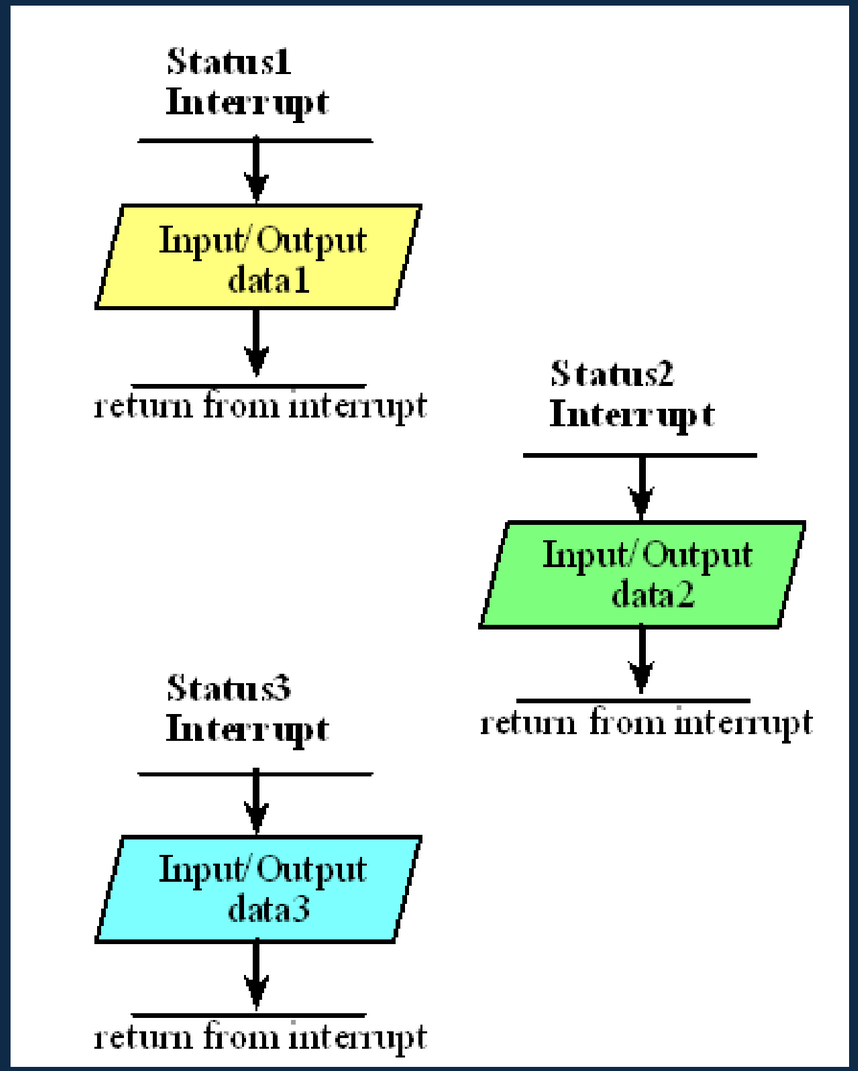
interrupts: what are they?

the polling vs interrupts:

Interrupts provide a better mechanism for managing events.

An interrupt is a signal which tells the CPU that an event has occurred. Upon receiving this signal, the CPU stops execution of application code, saves its state (registers)* , and then handles the event that issued the interrupt.

Interrupts are implemented in hardware, which means the exact mechanism in which they work depends on the used Microcontroller.



How do Interrupts work in atmega328p

For an Interrupt to be handled in atmega328p, three conditions must be met:

1. globalEnableBit must be set: The Global Interrupt bit is the I-bit (7th) in the status register SREG

2. enableBit must be set: Each source of interrupt (Timer, Pin, UART..) has an individual bit to enable its interrupt.

3. interruptFlag must be set: an interrupt flag is set when its source issues an interrupt, examples: UART module issues an interrupt to signal that it finished sending a byte; Pin configured to cause an interrupt toggles..etc

How do Interrupts work in atmega328p

If all three conditions are met, and an interrupt happens, the CPU pushes its state onto the stack, and retrieves the Interrupt Service Routine (ISR) address from the Interrupt Vector Table (IVT).
The Interrupt Vector Table holds the addresses to ISRs of different sources, and it lives in the first addresses of program memory.

11.1 Interrupt Vectors in ATmega328P

Table 11-1. Reset and Interrupt Vectors in ATmega328P

Vector No.	Program Address	Source	Interrupt Definition
1	0x0000	RESET	External pin, power-on reset, brown-out reset and watchdog system reset
2	0x002	INT0	External interrupt request 0
3	0x0004	INT1	External interrupt request 1
4	0x0006	PCINT0	Pin change interrupt request 0
5	0x0008	PCINT1	Pin change interrupt request 1
6	0x000A	PCINT2	Pin change interrupt request 2
7	0x000C	WDT	Watchdog time-out interrupt
8	0x000E	TIMER2 COMPA	Timer/Counter2 compare match A

How do Interrupts work in atmega328p

As programmers, we are supposed to configure registers to enable of interrupts globally and for individual sources that we wish to use. We also write the code of the ISR which is executed once the interrupt happens.

Notes about interrupts handling in atmega328p:

1. Interrupt flag is automatically cleared when entering the ISR (by hardware).
2. Interrupts are disabled (Global Interrupt Enable I-bit is cleared) when entering ISR and re-enabled when it is exited. This prevents Nested-Interrupts (may cause Stack Overflow). However, user software can enable Nested-Interrupts by writing logic 1 to I-bit.

Timers:

A Timer/counter is a module inside the MCU that is attached to the CPU clock (System Clock). It is used to count for certain durations of time, and generate digital signals with specific time requirements (PWM signals for example).

- We can use Timers to:

1. Generate Interrupts in certain time periods.

2. Measure elapsed time

3. Generate digital waveforms (such as PWM)

Timers: on what basis do they work?

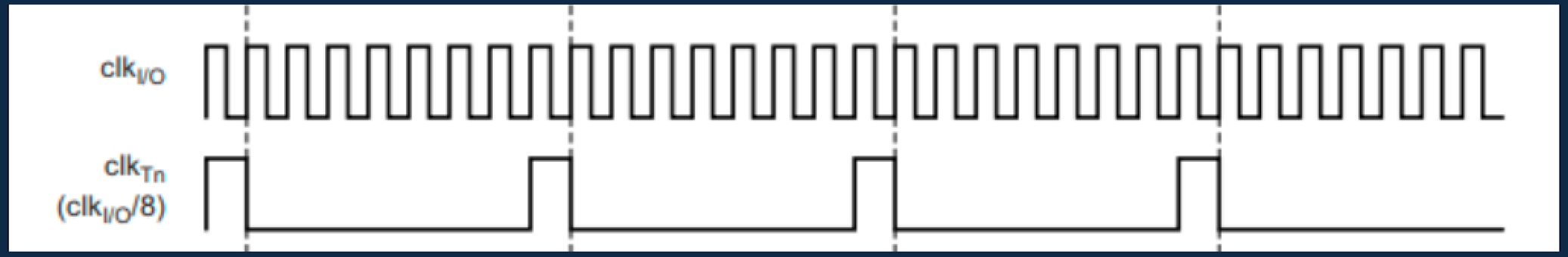
A Timer/counter uses the system clock or a prescaled version of it.

- At each rising edge of the Timer's clock, the Timer's Counter register is incremented. Hence, the timer can count for a **maximum time period** that is equal to: **Max value in Counter register * Timer's clock period**

- Timer uses a prescaler to derive its clock signal (with lower frequency) from the system signal:

16Mhz clk

2Mhz timer clk



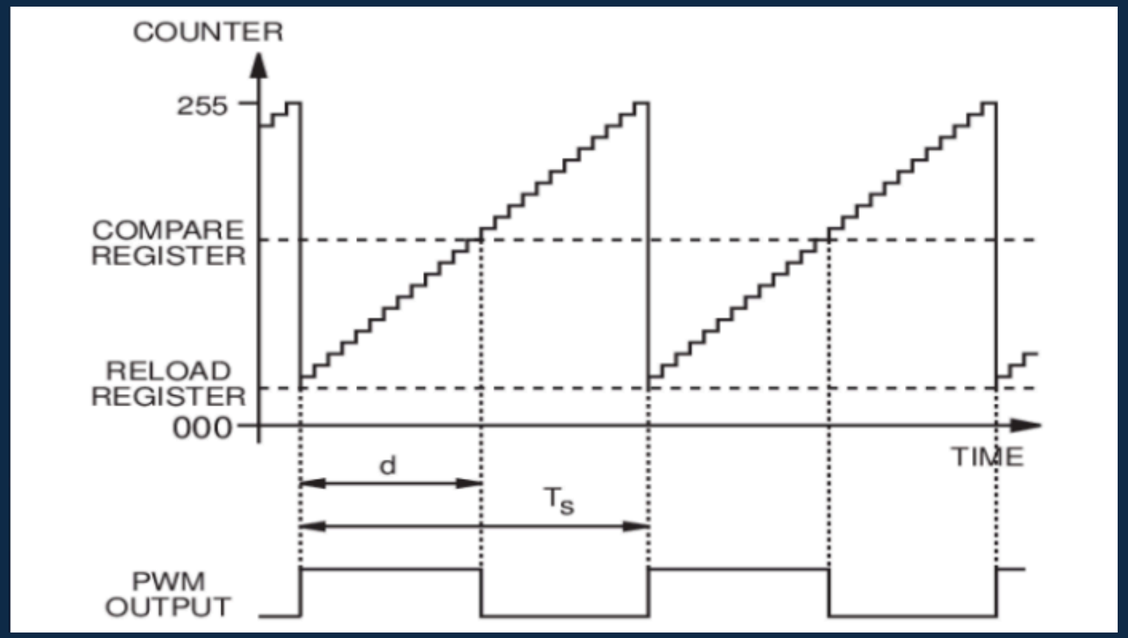
Timers, atmega328p’s timer0/2

Atmega328p has three timers: 2 of which are 8-bit (Timer0 and Timer2) and one of which is 16-bit (Timer1). Each Timer has 4 main modes: Normal, CTC, Fast PWM, and Phase Correct PWM.

- Timer0 can count from 0 -> 255, depending on which mode the timer is used with, it can:

1. Count for a duration of time and issue an interrupt using what’s called a Compare Unit.

2. Generate digital waveforms (such as PWM)



Atmega328p’s Timer1

- Timer1 is a 16-bit timer with 2 independent compare units.

- Generally speaking, Timer1 counts from BOTTOM -> TOP. These two values can either be 0 -> 65535(MAX) or other selected values.

Table 15-1. Definitions

Parameter	Definition
BOTTOM	The counter reaches the BOTTOM when it becomes 0x0000.
MAX	The counter reaches its MAXimum when it becomes 0xFFFF (decimal 65535).
TOP	The counter reaches the TOP when it becomes equal to the highest value in the count sequence. The TOP value can be assigned to be one of the fixed values: 0x00FF, 0x01FF, or 0x03FF, or to the value stored in the OCR1A or ICR1 register. The assignment is dependent of the mode of operation.

- Timer1 registers are used to configure its mode of operation, clock signal..etc

1. **TCNT1 (Timer Counter):** holds the counter value, and is incremented at every rising edge of Timer1's clock.

2. **OCR1A/OCR1B (Output Compare Register A/B)**: holds value of comparison. Each compare unit has its own output compare register (2 units 2 registers A and B). If configured, an interrupt can be issued each time a match occurs between TCNT1 and OCR1A or OCR1B.

3. **TIFR1 (Timer Interrupt Flag Register):** when Timer1 issues an interrupt, depending on which interrupt it issued, a flag is set by hardware in this register.

TIFR1 – Timer/Counter1 Interrupt Flag Register

Bit	7	6	5	4	3	2	1	0	
0x16 (0x36)	–	–	ICF1	–	–	OCF1B	OCF1A	TOV1	TIFR1
Read/Write	R	R	R/W	R	R	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

input capture flag

output compare flags

timer overflow flag

4 . **TCCR1A/TCCR1B (Timer/Counter Control Register):** Timer1 has two control registers, each register control certain options (Refer to atemga328p datasheet for the use of each bit p132 133)

16.11.1 TCCR1A – Timer/Counter1 Control Register A

Bit	7	6	5	4	3	2	1	0	
(0x80)	COM1A1	COM1A0	COM1B1	COM1B0	–	–	WGM11	WGM10	TCCR1A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

16.11.2 TCCR1B – Timer/Counter1 Control Register B

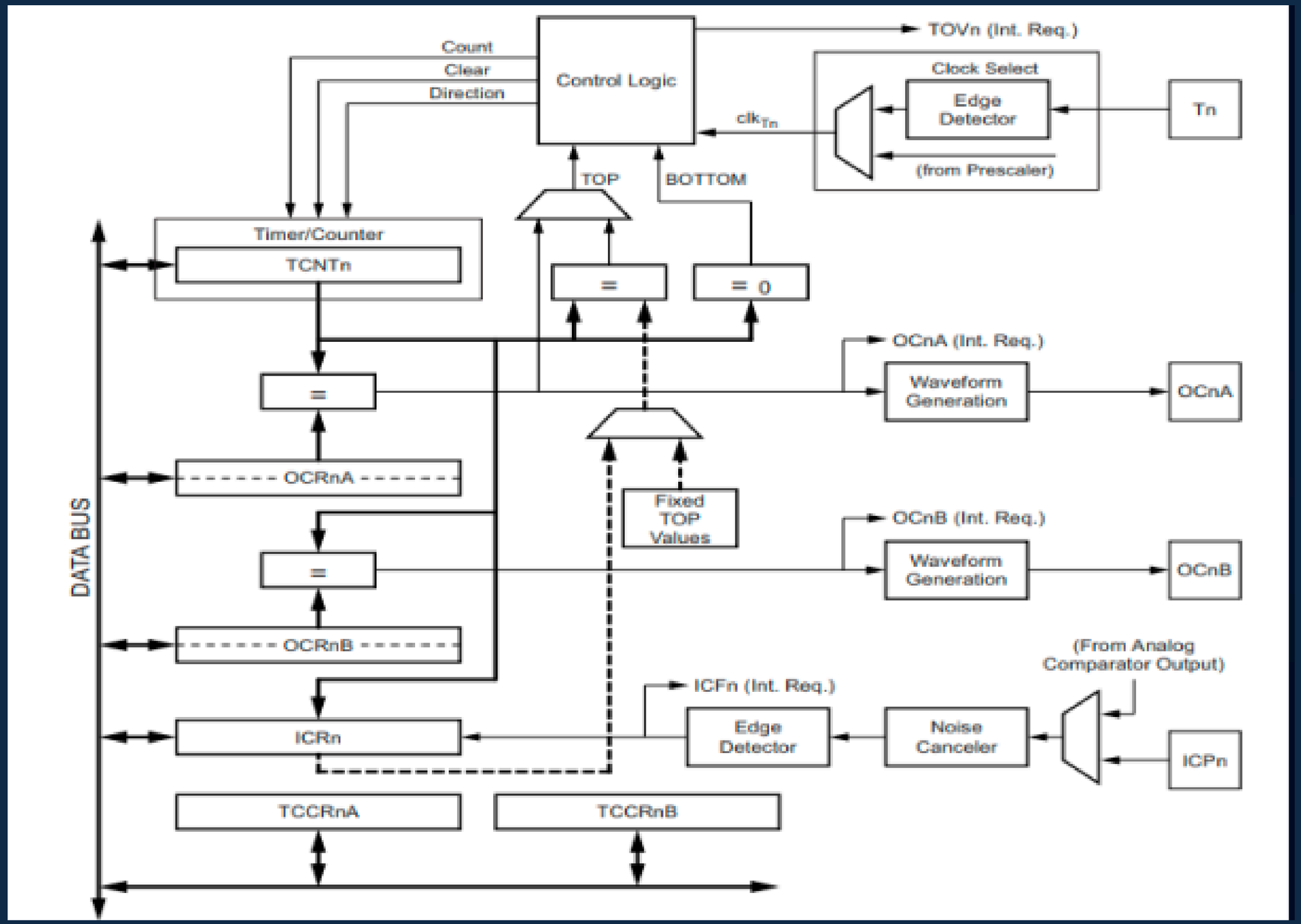
Bit	7	6	5	4	3	2	1	0	
(0x81)	ICNC1	ICES1	–	WGM13	WGM12	CS12	CS11	CS10	TCCR1B
Read/Write	R/W	R/W	R	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

5. **TIMSK1 (Timer Interrupt Mask):** **writing 1 to a bit in this register enables a source of interrupt from Timer1**

16.11.8 TIMSK1 – Timer/Counter1 Interrupt Mask Register

Bit	7	6	5	4	3	2	1	0	
(0x6F)	–	–	ICIE1	–	–	OCIE1B	OCIE1A	TOIE1	TIMSK1
Read/Write	R	R	R/W	R	R	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Timer 1's architecture :



Timer1's normal mode:

A timer that just counts then overflows

Counting time:

- delay time = number of ticks * Timer’s clock period

- delay time = number of ticks * (prescalar / F_CPU)

hence :

- number of ticks = (delay time * F_CPU / prescalar) - 1

Timer1's normal mode:

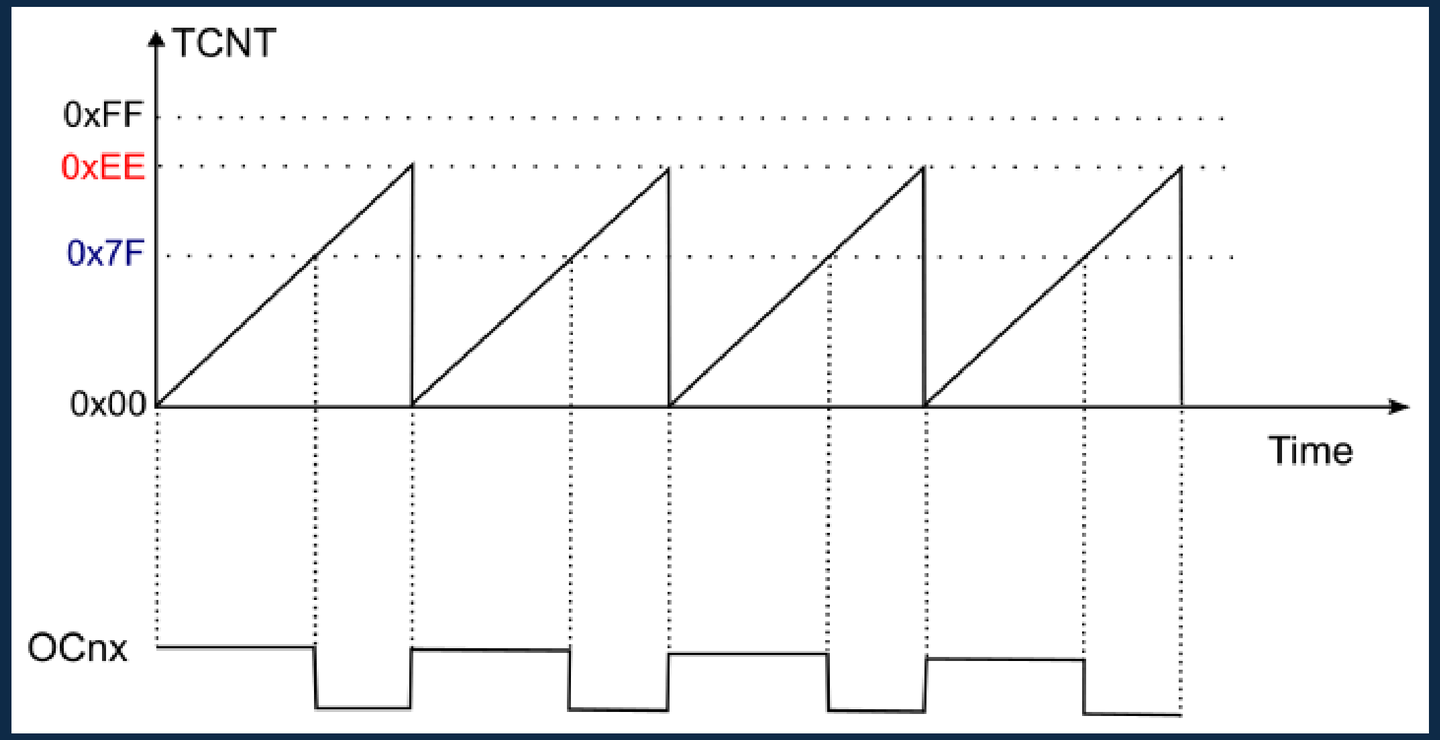
```
#define PINMASK(x) (1U << x)
#define CPU_F 16000000L

void setup() {
  DDRB |= PINMASK(5);
  // Set Timer1 to simply count, no compare no waveform generation
  TCCR1A = 0x00;
  // Timer Interrupt Mask. Sets interrupt for timer overflow
  TIMSK1 = (1 << TOIE1);
  // Make Timer1 start counting from 49910 instead of 0
  TCNT1 = 0xFFFF - (CPU_F/1024);
  sei(); // enable interrupts
  // CS = Clock Select | Timer/Counter Control Register (for timer 1)
  // this line sets prescaler to 1024 | 16MHz/1024 = 15625Hz
  TCCR1B = (1 << CS10) | (1 << CS12);
}
void loop() {
  while(1);
}
```

Timer1's normal mode:

```
// Timer 1 automatically resets to 0 after overflow,
// so we force it to start from 49910 instead
ISR(TIMER1_OVF_vect){
    PORTB ^= PINMASK(5);
    TCNT1 = 0xFFFF - (CPU_F/1024);
}
```

Timer1's PWM mode :



TOP value (in red) sets the frequency
Compare value (in blue) sets the duty cycle

SG90 Servo:

- Commonly used in robotics and Arduino projects.
- Controlled by a 50Hz PWM signal, with a rotation range of 0° to 180°.
- The motor moves to a position based on the pulse width.
  - 500 µs → 0° (full counterclockwise)
  - 1500 µs → 90° (middle position)
  - 2400 µs → 180° (full clockwise)

Rotational Range:	180°
Pulse Cycle:	ca. 20 ms
Pulse Width:	500-2400 µs

PWM and Servos :

For a PWM signal with 50HZ we need:

1) set the mode to mode 15:

Mode	WGM13	WGM12 (CTC1)	WGM11 (PWM11)	WGM10 (PWM10)	Timer/Counter Mode of Operation	TOP	Update of OCR1x at	TOV1 Flag Set on
13	1	1	0	1	(Reserved)	–	–	–
14	1	1	1	0	Fast PWM	ICR1	BOTTOM	TOP
15	1	1	1	1	Fast PWM	OCR1A	BOTTOM	TOP

The value of OCR1A defines the frequency of the PWM signal.
The Bits WGM11:10 are found in TCCR1A
The Bits WGM12:13 are found in TCCR1B

PWM and Servos :

2) set the mode to a non-inverting Fast PWM 15:

Table 16-2. Compare Output Mode, Fast PWM[1]

COM1A1/COM1B1	COM1A0/COM1B0	Description
0	0	Normal port operation, OC1A/OC1B disconnected.
0	1	WGM13:0 = 14 or 15: Toggle OC1A on Compare Match, OC1B disconnected (normal port operation). For all other WGM1 settings, normal port operation, OC1A/OC1B disconnected.
1	0	Clear OC1A/OC1B on Compare Match, set OC1A/OC1B at BOTTOM (non-inverting mode)
1	1	Set OC1A/OC1B on Compare Match, clear OC1A/OC1B at BOTTOM (inverting mode)

These bits are found in TCCR1A

PWM and Servos :

3- The clock source and the presclar:

Table 16-5. Clock Select Bit Description

CS12	CS11	CS10	Description
0	0	0	No clock source (Timer/Counter stopped).
0	0	1	$clk_{I/O}/1$ (No prescaling)
0	1	0	$clk_{I/O}/8$ (From prescaler)
0	1	1	$clk_{I/O}/64$ (From prescaler)
1	0	0	$clk_{I/O}/256$ (From prescaler)
1	0	1	$clk_{I/O}/1024$ (From prescaler)
1	1	0	External clock source on T1 pin. Clock on falling edge.
1	1	1	External clock source on T1 pin. Clock on rising edge.

These bits are found in TCCR1B

PWM and Servos :

3- Enable the interrupt :

TIMSK1 – Timer/Counter1 Interrupt Mask Register

Bit	7	6	5	4	3	2	1	0	
(0x6F)	–	–	ICIE1	–	–	OCIE1B	OCIE1A	TOIE1	TIMSK1
Read/Write	R	R	R/W	R	R	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Whenever the counter reaches the TOP value (TCNT1 = OCR1A) an output compare match interrupt is fired and TCNT1 is reset.

PWM and Servos :

Counting time:

- delay time = number of ticks * Timer’s clock period
- delay time = number of ticks * (prescalar / F_CPU)

hence :

- number of ticks = (delay time * F_CPU / prescalar) - 1

Let’s configure a Timer - PWM mode :

baremetalsg90.ino

```
#include<stdint.h>
#define PINMASK(x) (1U << x)
#define FCPU 16000000L

uint16_t PWM = 500;
volatile uint8_t delay_count = 0 ;

void setup (){
    // The Timer1 Output Compare Unit B (OC1B) is mapped to PB2 (pin 10) --  set as OUTPUT
    DDRB |= PINMASK(2);
    // SELECT THE MODE TO FAST PWM (15) AT OC1B ; BITS WGM13:12 ALL HIGH
    TCCR1A |= (1 << COM1B1) | (1 << WGM11) | (1<<WGM10) ;
    TCCR1B |= (1<<WGM12)| (1<< WGM13) ;
    // TOP VALUE OF THE TIMER (DEFINEs THE FREQUENCY)
    OCR1A = 4999 ;
    //RESET THE COUNTER COUNTER TO 0
    TCNT1 = 0 ;
    // ENABLE THE OUTPUT COMPARE MATCH INTERRUPT
    TIMSK1 |= (1<< OCIE1A);
    //ADJUST THE DUTY CYCLE
    OCR1B = (PWM/1000000)* FCPU /64 -1 ;
    // CONFIGURE THE PRESCALAR TO 64 -- F_timer = 16MHZ/64
    TCCR1B |= (1<<CS11)|(1<<CS10) ;
}
```

Let’s configure a Timer - PWM mode :

```
void loop(){

    for (uint8_t i=0 ; i<= 180 ; i+= 10){
        servo_write(i);
        Delay(20);
    }

    for(uint8_t i=180 ; i>0 ; i-=10){
        servo_write(i);
        Delay(20);
    }
}
void servo_write(int angle){
    cli(); // disable global interrupts
    PWM = map(angle , 0, 180, 500, 2400); // map the angle to the pulse width
    sei(); // enable global interrupts
}

ISR(TIMER1_COMPA_vect){
    // OCR1B= PULSE WIDTH IN SEC * TIMER FREQUENCY -1
    OCR1B = (PWM/1000000.0)*FCPU /64-1 ;
    delay_count++ ;
}
void Delay(unsigned int n ) { // delay for n*20ms
    cli();
    delay_count = 0 ;
    sei();
    while (delay_count <n) ;
}
```

THANK YOU!